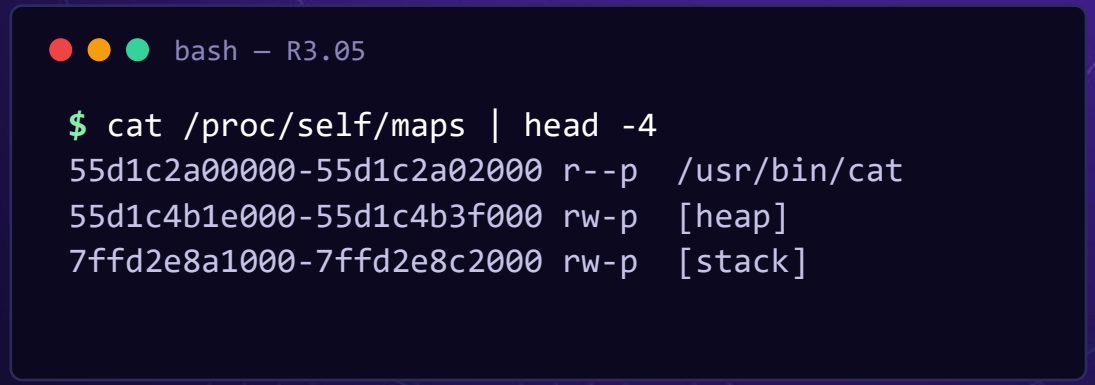
lamae

BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 07a — La mémoire virtuelle (1/2)



Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

Au programme

01

Le problème

Plusieurs programmes, une mémoire trop petite et chère.

02

Les débuts

Overlays et va-et-vient (swapping), suivi de la mémoire libre.

03

La MMU

Adresses logiques, linéaires, physiques ; niveaux de privilège.

04

La segmentation

Le principe, l'exemple du 8086, et pourquoi Linux s'en passe.

Suite au CM 07b : la pagination, la traduction d'adresse, les défauts de page et le swap sous Linux.

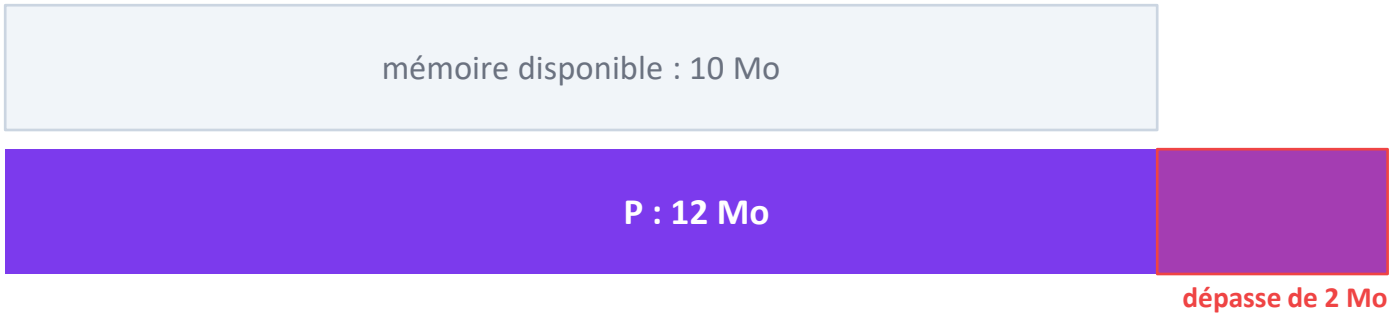
01

Le problème

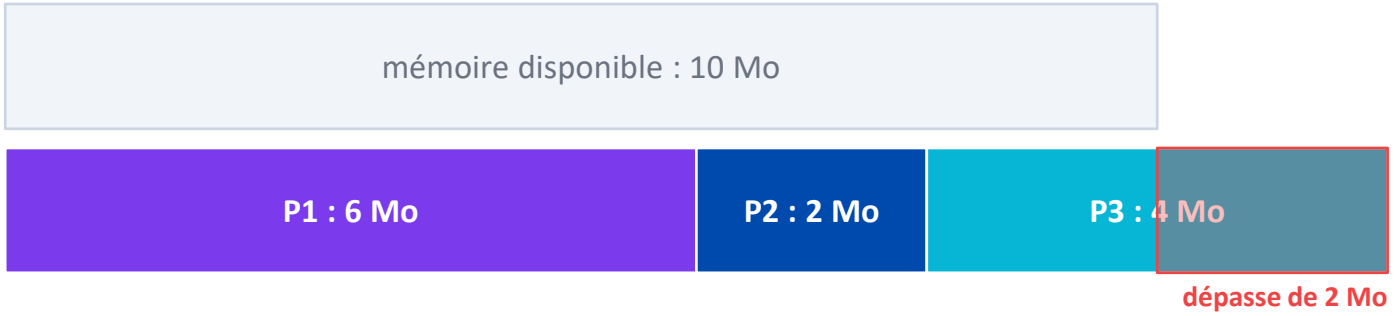
Faire tenir plus de programmes que la mémoire ne le permet

Trop de programmes, pas assez de mémoire

Monoprogrammation



Multiprogrammation



Le constat

La mémoire vive est rapide mais chère. La mémoire secondaire (disque, SSD) est lente mais bon marché.

L'idée

Utiliser la mémoire secondaire « comme » de la RAM : si A veut s'exécuter et qu'il n'y a plus de place, un morceau d'un autre programme part sur le disque et un morceau de A prend sa place.

Les objectifs de la gestion mémoire

01

Partager

Plusieurs processus en mémoire en même temps, et parfois du code commun.

02

Allouer

Donner des blocs de mémoire aux processus, et les reprendre.

03

Protéger

Un processus ne doit pas lire ni écrire la mémoire d'un autre, ni celle du noyau.

04

Optimiser

Tirer le maximum de la mémoire disponible, et l'étendre avec le disque.

02

Les débuts

Overlays et va-et-vient

Quand la mémoire se comptait en kilo-octets

« 640 Ko devraient suffire à tout le monde. »

Citation attribuée à Bill Gates (1981)… qui a toujours nié l'avoir dite. Elle est très probablement apocryphe.

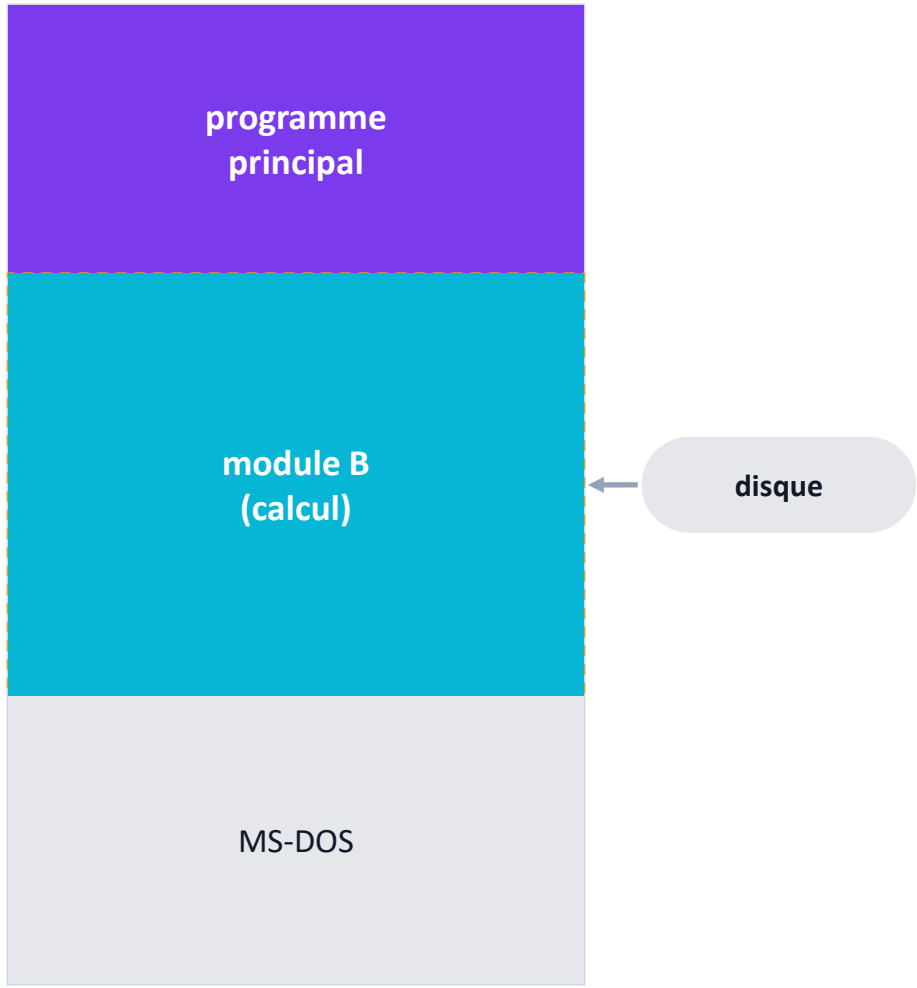
Le contexte

Le processeur Intel 8088 des premiers PC adressait 1 Mo. Sous MS-DOS, seuls les 640 premiers Ko étaient disponibles pour les programmes : la « mémoire conventionnelle ».

Sinclair ZX Spectrum 1982	**48 Ko**
Commodore 64 1982	**64 Ko**
Oric-1 1983	**48 Ko**
Votre téléphone aujourd'hui	**8 à 16 Go**

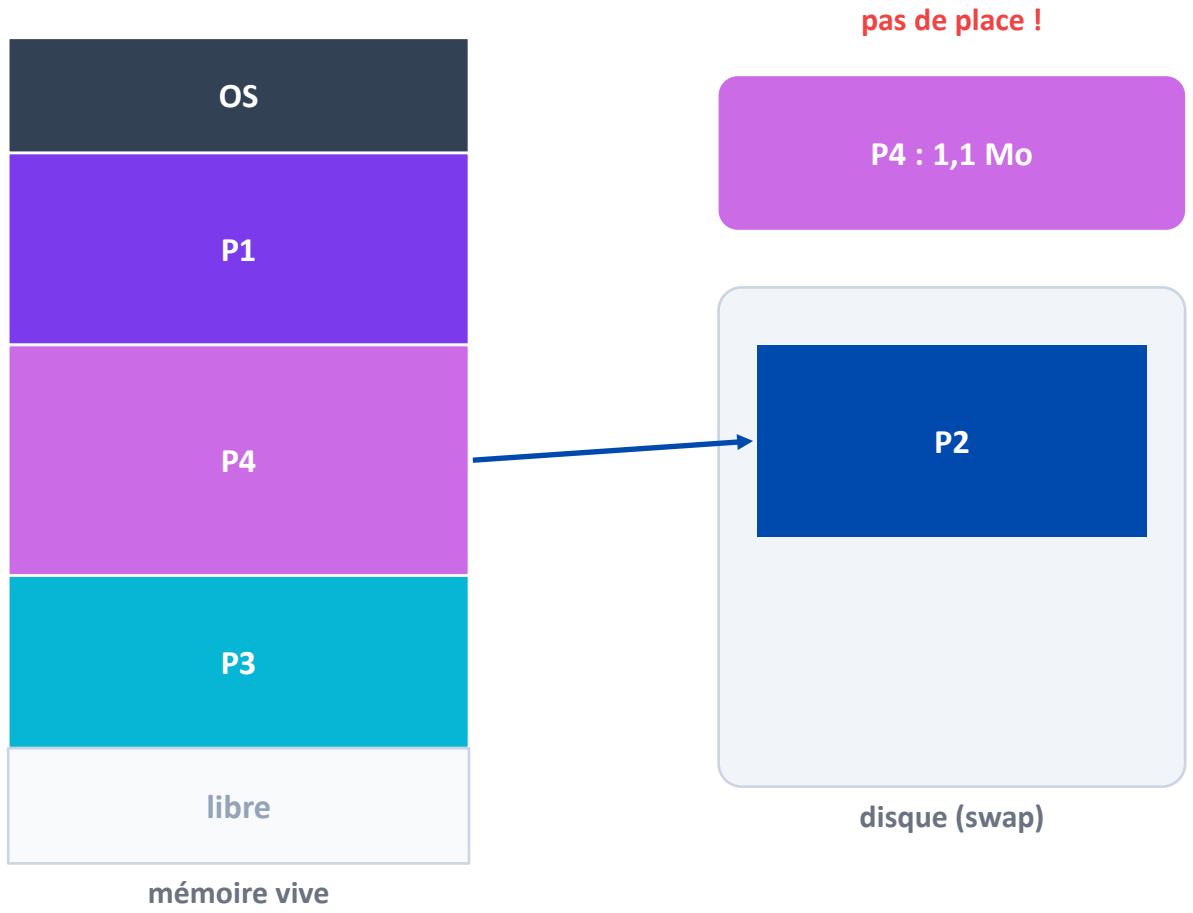
Les segments de recouvrement (overlays)

- Le programme est trop gros pour la mémoire : le programmeur le découpe en blocs
- Une partie reste toujours en mémoire (le programme principal)
- Les autres blocs se succèdent dans la même zone : on charge B par-dessus A quand A n'est plus utile
- Technique complexe, abandonnée… sauf dans certains systèmes embarqués à très peu de mémoire



Le va-et-vient (swapping)

- Le système déplace des processus entiers entre la mémoire et le disque
- Un processus est soit entièrement en mémoire, soit entièrement sur le disque
- Utilisé sous DOS et Windows 3.1, et dans les premiers UNIX



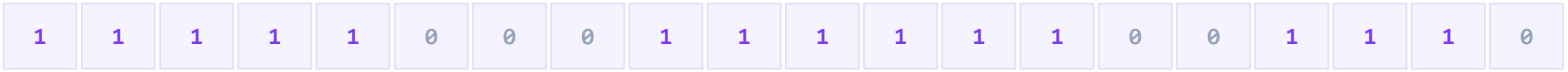
Les limites : les transferts de processus entiers sont lents ; un programme plus grand que la mémoire ne peut toujours pas s'exécuter ; et la mémoire se fragmente.

Savoir où est la mémoire libre

La mémoire est découpée en unités d'allocation. Le système doit savoir lesquelles sont occupées.

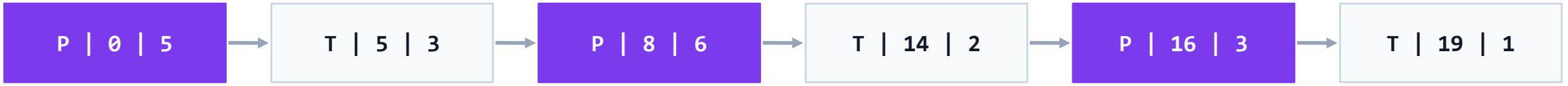


1. Table de bits



1 bit par unité : 1 = occupée, 0 = libre. Simple, mais trouver k unités libres consécutives oblige à parcourir la table.

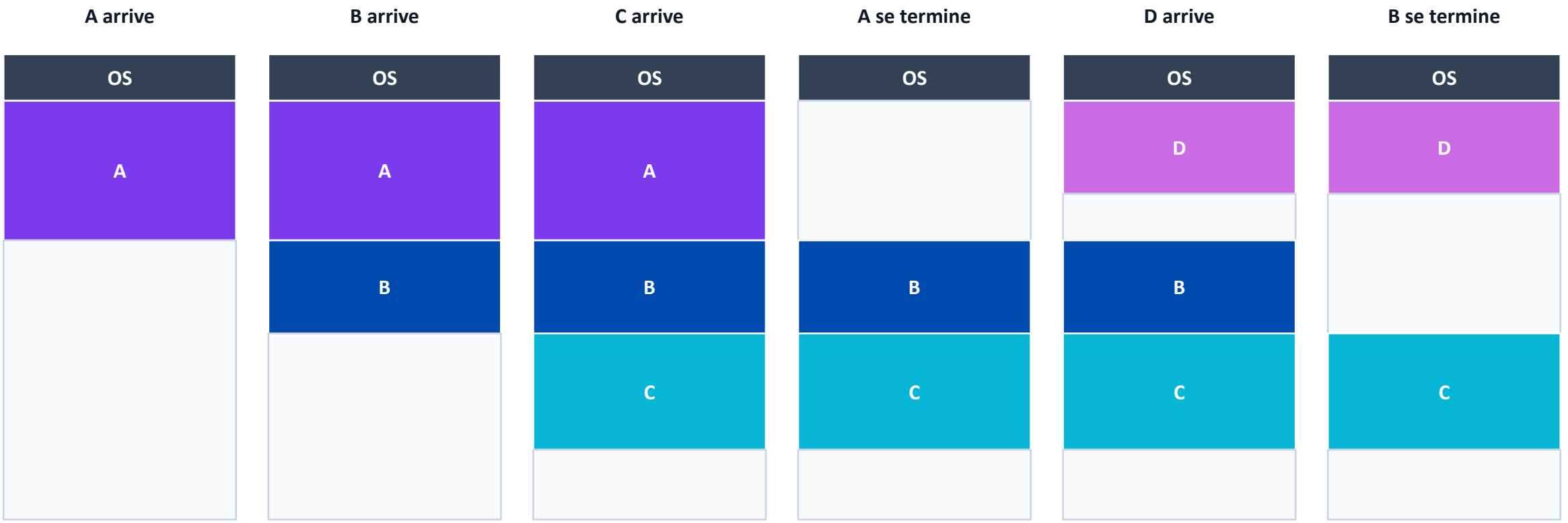
2. Liste chaînée



P = processus, T = trou (hole) ; adresse de départ ; taille. Fusionner les trous voisins quand un processus se termine.

Placer un nouveau processus : première zone libre assez grande (first fit), zone suivante (next fit), meilleure zone (best fit), plus grande zone (worst fit).

Au fil du temps : la fragmentation



À la fin : 4,5 unités libres au total, mais en deux trous (3 et 1,5). Un processus de 4 unités ne peut pas entrer. Remède : compacter la mémoire… très coûteux.

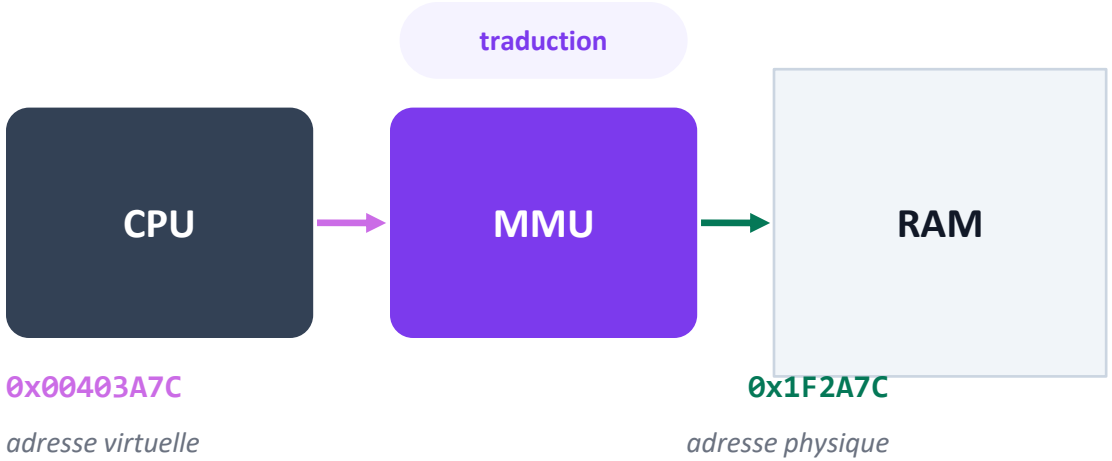
03

La MMU

L'unité de gestion mémoire du processeur

La MMU (Memory Management Unit)

- Intégrée à presque tous les processeurs actuels, utilisée par tous les systèmes modernes (Linux, Windows, macOS)
- Elle traduit chaque adresse utilisée par un programme en une adresse physique, à chaque accès
- Elle protège : un accès interdit déclenche une exception (le noyau envoie SIGSEGV)



Unité de segmentation

adresse logique → linéaire

Unité de pagination

adresse linéaire → physique

Unité de protection

droits, niveaux de privilège

De l'adresse logique à l'adresse physique

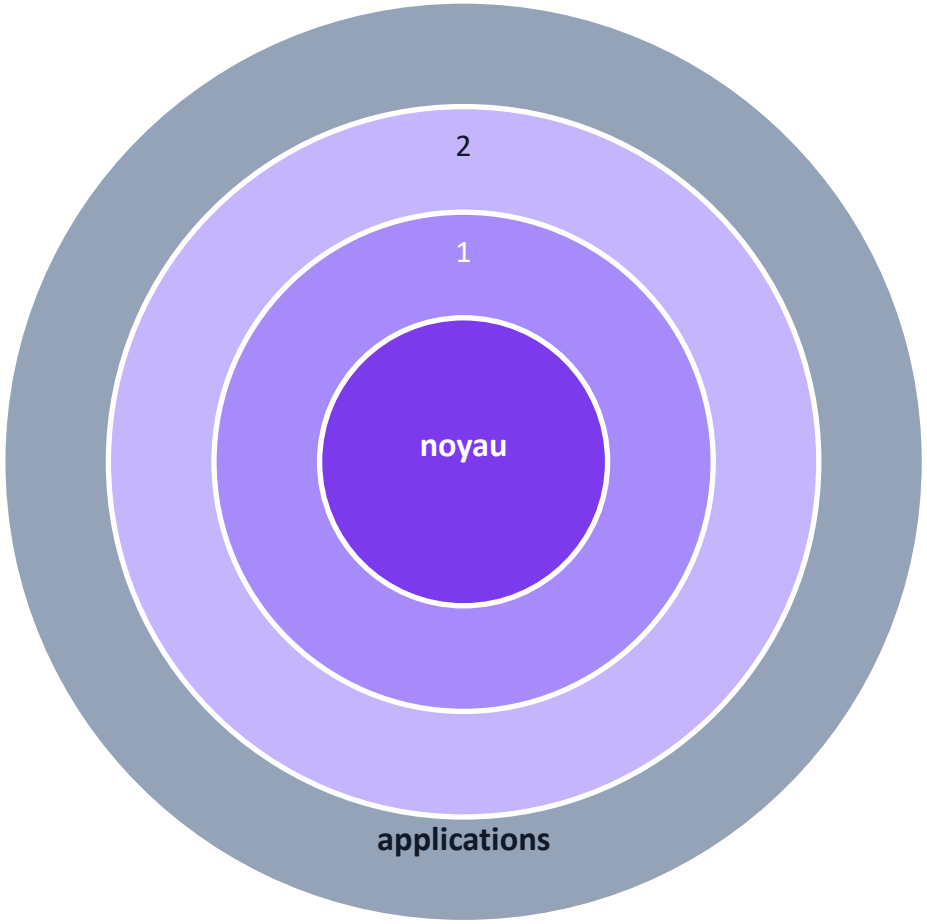


Sous Linux : tous les segments commencent à l'adresse 0 et couvrent tout l'espace (modèle « plat »). L'adresse logique est donc égale à l'adresse linéaire : seule la pagination traduit vraiment.

Ce qu'affiche printf("%p", &x) est une adresse virtuelle. Deux processus peuvent afficher la même adresse pour des données différentes : chacun a son propre espace virtuel.

Les niveaux de privilège

- Depuis le 80286, les processeurs Intel ont un mode protégé avec 4 niveaux de privilège (anneaux 0 à 3)
- Le niveau courant décide quelles instructions et quelles zones mémoire sont permises
- Linux n'utilise que 2 anneaux : 0 pour le noyau, 3 pour les applications
- C'est le mode noyau et le mode utilisateur du CM 03a



anneaux 1 et 2 : non utilisés par Linux

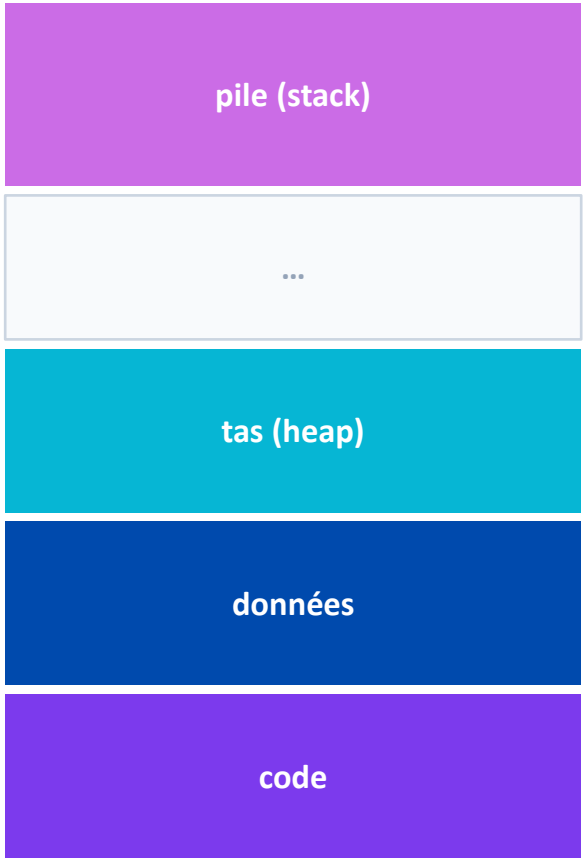
04

La segmentation

Le principe, l'exemple du 8086, et ce qu'il en reste

Le principe de la segmentation

- Découper l'espace d'un programme en segments logiquement indépendants : code, données statiques, tas, pile
- Chaque segment a une base, une taille et des droits (lecture, écriture, exécution)
- Une adresse logique = (segment, décalage dans le segment)
- Un décalage plus grand que la taille du segment : erreur de segmentation. C'est l'origine du nom « segmentation fault » !



Aujourd'hui, ce découpage est préparé par le compilateur et l'éditeur de liens.

Exemple : le 8086 (1978)

- Bus d'adresse de 20 bits : 1 Mo adressable
- Registres de 16 bits : 64 Ko seulement…
- Solution : 4 registres de segment (CS code, DS données, SS pile, ES extra)

```
adresse physique = segment × 16 + décalage
```

Au démarrage, CS = 0xFFFF et IP = 0x0000 : le processeur exécute l'instruction de l'adresse 0xFFFF0, tout en haut du premier Mo (le BIOS). Les PC actuels démarrent encore ainsi, en mode réel.

`DS = 0xFB0A, décalage = 0x0005`

segment	`0xFB0A`	
décalé de 4 bits	`0xFB0A0`	*× 16*
+ décalage	`0x00005`	
adresse physique	`0xFB0A5`	

DS	segment:décalage	adresse physique
`0xFB0A`	DS:0x0005	0xFB0A5
`0xF000`	DS:0xB0A5	0xFB0A5
`0xF203`	DS:0x9075	0xFB0A5

Plusieurs couples désignent la même case : les segments peuvent se chevaucher.

Et aujourd'hui ?

Mode réel

Le modèle du 8086, conservé pour la compatibilité : segments de 64 Ko, 1 Mo adressable, aucune protection. Utilisé seulement au démarrage.

Mode segmenté

Segments de taille variable, décrits par des descripteurs (base, taille, type, privilège). Puissant, mais lourd.

Mode plat

Un seul grand espace contigu : les segments couvrent tout. C'est le mode utilisé par Linux, Windows et macOS.

Pourquoi Linux utilise si peu la segmentation

- Si tous les processus partagent les mêmes segments, le travail de la MMU est bien plus simple.
- Linux tourne sur de nombreuses architectures (ARM, RISC-V…) qui n'ont presque pas de segmentation.
- En x86-64, seuls les registres FS et GS servent encore : ils pointent sur les données propres à chaque thread.

05

Pile et tas

Où vivent vos variables

Deux zones dynamiques

La pile (stack)

Gérée automatiquement à chaque appel de fonction : paramètres, adresse de retour, variables locales. Fonctionne en LIFO (dernier entré, premier sorti). Taille limitée (8 Mo par défaut sous Linux).

Le tas (heap)

Géré par le programmeur avec malloc et free (CM 01b). Les blocs vivent jusqu'à leur libération. Grande taille, mais risque de fuites et de fragmentation.

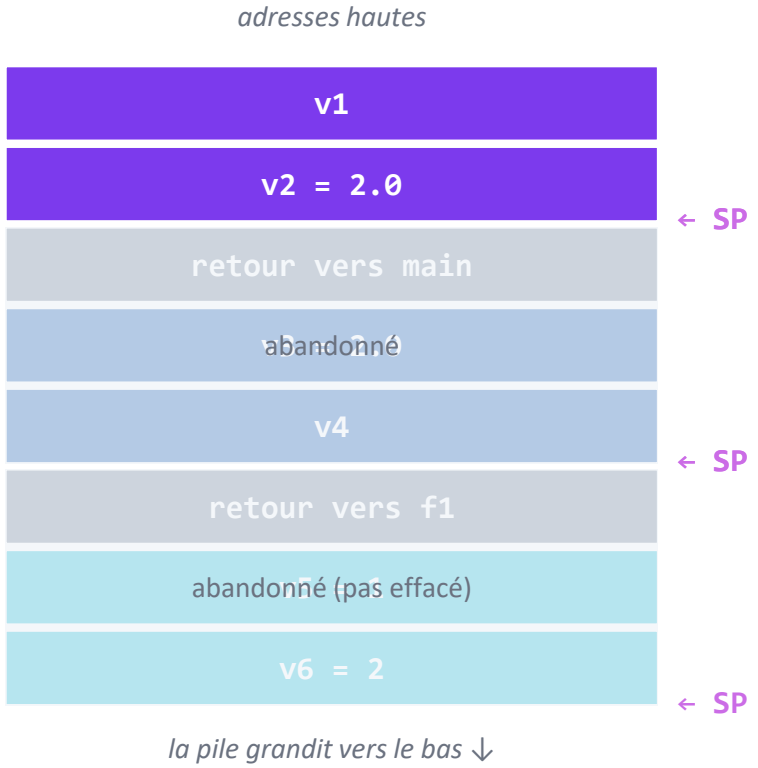
Registre (x86-64)	Rôle
`rsp (SP)`	Stack Pointer : sommet courant de la pile
`rbp (BP)`	Base Pointer : base du contexte de la fonction en cours
`rip (IP)`	Instruction Pointer : prochaine instruction à exécuter

Sur x86, la pile grandit vers les adresses basses.

La pile au fil des appels

pile.c

```c
short f2(char v5) {
    short v6 = 2;
    return v6;
}
void f1(float v3) {
    short v4;
    v4 = f2(1);
}
int main(void) {
    char v1;
    float v2 = 2.0;
    f1(v2);
    return 0;
}
```



Rien n'est effacé au retour : seul SP remonte. Lire une variable locale non initialisée, c'est lire ces restes.

Ce que produit le compilateur

main.c

```c
int main(void)
{
    float x = 8.0;
    return 0;
}
```

gcc -S -masm=intel main.c

```
main:
    push rbp                ; sauve le BP de l'appelant
    mov  rbp, rsp           ; BP = sommet : nouveau contexte
    mov  eax, 0x41000000    ; 8.0 en IEEE 754
    mov  [rbp-4], eax       ; x est rangé à BP - 4
    mov  eax, 0             ; valeur de retour dans eax
    pop  rbp                ; restaure le BP de l'appelant
    ret                     ; dépile l'adresse de retour
```

À retenir
Les variables locales sont repérées par rapport à BP. La valeur de retour passe par un registre (eax). ret reprend l'adresse empilée par call.

`8.0 = 0x41000000 :` signe +, exposant 130 − 127 = 3, mantisse 1,0 → $1{,}0 \times 2^3$

Essayez : gcc -S -O0 -masm=intel main.c, puis ouvrez main.s.

Le débordement de pile

recursion.c

```c
unsigned long f(unsigned long n)
{
    char tampon[1024];        // 1 Ko par appel
    tampon[0] = (char) n;
    return f(n + 1) + tampon[0];   // jamais de fin !
}
```

terminal

```
$ ulimit -s          # taille de la pile, en Ko
8192
$ ./recursion
Erreur de segmentation (core dumped)
$ ulimit -s 16384    # l'agrandir pour ce shell
```

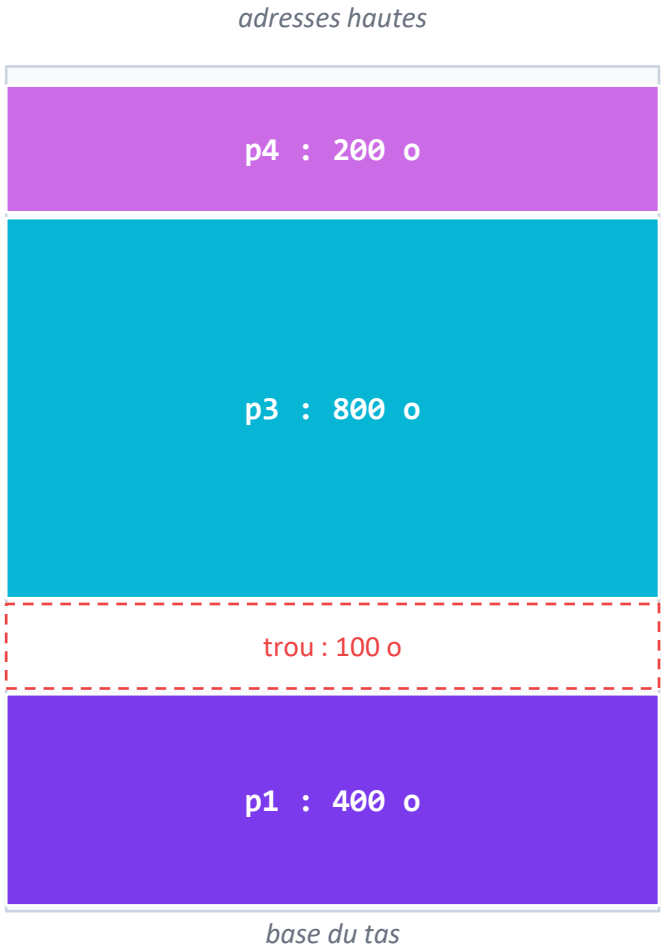
- Chaque appel empile un nouveau contexte : une récursion infinie remplit la pile
- Au-delà de la limite, la page suivante n'est pas autorisée : le noyau envoie SIGSEGV (stack overflow)
- getrlimit / setrlimit(RLIMIT_STACK) : lire ou changer la limite en C
- Éviter les très gros tableaux locaux : préférer malloc

Le tas au fil des malloc et free

tas.c

```
char *p1 = malloc(400);
char *p2 = malloc(100);
char *p3 = malloc(800);
free(p2);
char *p4 = malloc(200);
```

Fragmentation du tas : le bloc de 200 octets ne tient pas dans le trou de 100. Le trou restera inutilisé tant qu'une petite allocation ne viendra pas le remplir. Un programme qui alloue et libère beaucoup consomme ainsi plus de mémoire que nécessaire.



malloc sous le capot

brk / sbrk

Pour les petites allocations, malloc agrandit le tas en déplaçant sa limite (le « program break »).

mmap

Pour les grosses allocations (128 Kio et plus par défaut), malloc demande une zone séparée au noyau ; free la rend aussitôt.

Surallocation

Le noyau accorde souvent plus de mémoire qu'il n'en a : les pages ne sont réservées qu'à la première écriture (CM 01b et CM 07b).

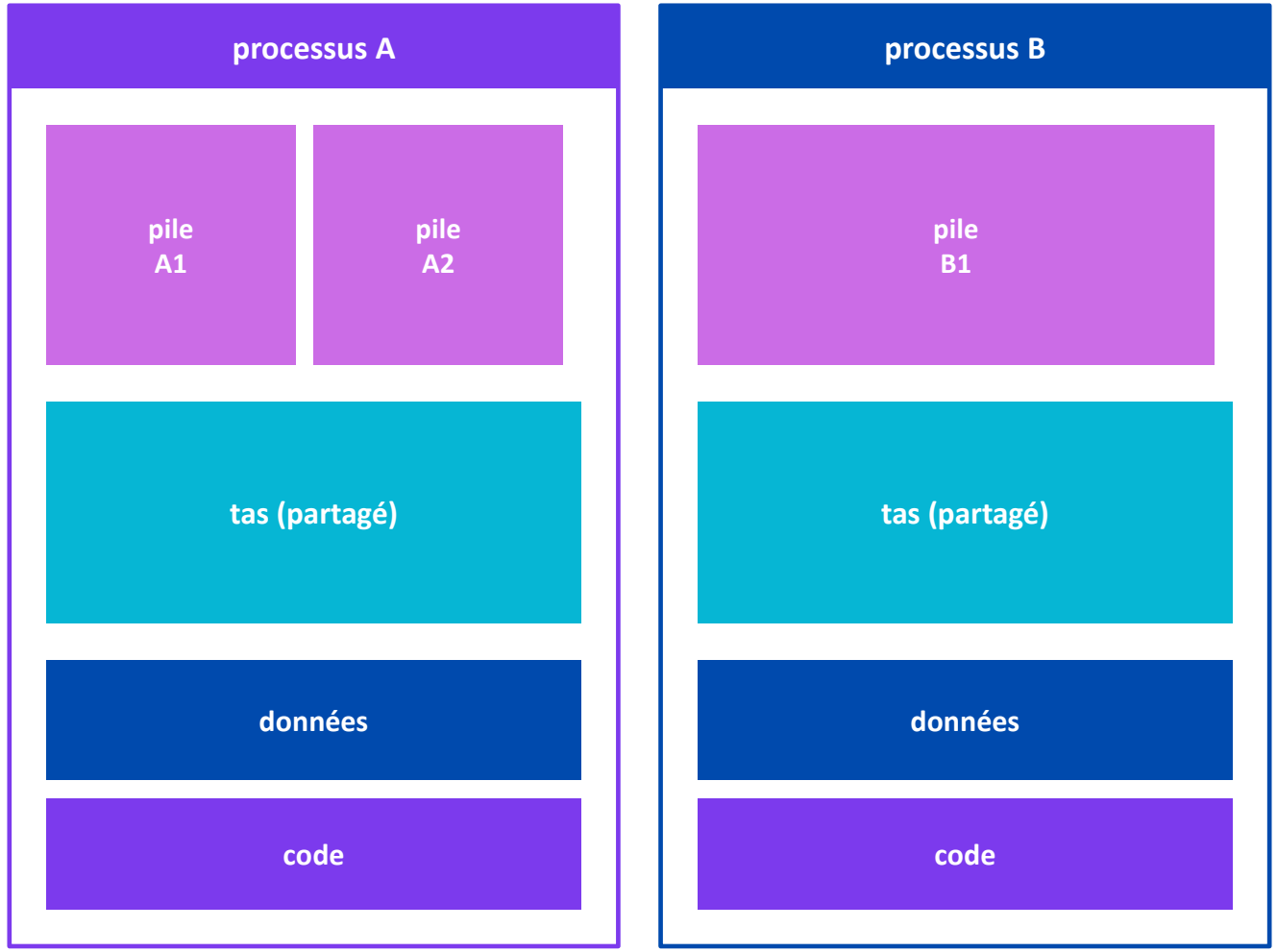
observer les appels système de malloc

```
$ strace -e trace=brk,mmap ./tas
brk(NULL)                               = 0x55d1c4b1e000
brk(0x55d1c4b3f000)                     = 0x55d1c4b3f000
mmap(NULL, 1052672, PROT_READ|PROT_WRITE, ...) = 0x7f3a9c2fe000
```

Sur certains microcontrôleurs, free ne rend jamais la mémoire : lisez la documentation de votre chaîne de compilation.

Pile, tas et threads

- Le tas appartient au processus : il est partagé par tous ses threads
- Chaque thread a sa propre pile (ses appels de fonctions, ses variables locales)
- Deux processus n'ont rien en commun : ni tas, ni pile



Conséquence (séance 8) : les threads d'un même processus peuvent se gêner sur les données du tas et les globales : il faut des mutex.

SYNTHÈSE

À retenir

01

Le besoin

Partager, allouer, protéger, optimiser la mémoire, et l'étendre avec le disque.

02

Les débuts

Overlays (gérés par le programmeur), va-et-vient de processus entiers : lents et fragmentés.

03

La MMU

Traduit adresse virtuelle → physique à chaque accès, et protège. Linux : anneaux 0 et 3.

04

Segmentation

8086 : segment × 16 + décalage. Linux utilise le modèle plat : seule la pagination traduit.

05

La pile

Un contexte par appel (retour, BP, locales). Récursion infinie → SIGSEGV.

06

Le tas

malloc / free, via brk et mmap. Fragmentation possible. Partagé par les threads.

Ressources pédagogiques

Explorer sur votre machine

```
$ cat /proc/self/maps     # l'espace d'un
processus
$ pmap -x <pid>
$ ulimit -a
$ man 2 brk
$ man 3 malloc
$ gcc -S -masm=intel f.c
```

/proc/self/maps montre les zones de code, de données, le tas, la pile et les bibliothèques du processus qui le lit.

Lectures

- A. Tanenbaum, Systèmes d'exploitation : « Gestion de la mémoire »
- R. et A. Arpaci-Dusseau, Operating Systems: Three Easy Pieces : chapitres 13 à 17 (espaces d'adressage, segmentation, espace libre)
- Intel, Software Developer's Manual, vol. 3 : modes réel, protégé et 64 bits

lamae

Questions ?

PROCHAINE SÉANCE

CM 07b — La mémoire virtuelle (2/2)

pagination · table des pages · TLB · défauts de page · swap

Christophe Cruz · christophe.cruz@ube.fr